

Diseño de un sistema basado en el microprocesador Z80 con interfaz de memoria y puertos

Juana Janet García Sánchez

Profesor: Jose Luis Barbosa Pacheco | Grupo: 1711
Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán

Resumen

El presente trabajo describe el diseño y simulación de un sistema computacional basado en el microprocesador Z80. El hardware cuenta con una capacidad de 16 Kbytes de memoria de solo lectura (EEPROM 28C256) para almacenar código, 16 Kbytes de memoria de acceso aleatorio (dos memorias SRAM 6264) para datos, y 6 puertos paralelos de entrada/salida implementados mediante dos interfaces periféricas programables (PPI 8255). El diseño incluye la elaboración del diagrama electrónico en KiCad y la programación del sistema mediante lenguaje BASIC ejecutado sobre un intérprete ensamblado en Z80 workbench, logrando una interfaz interactiva que solicita, valida y contabiliza cadenas de caracteres.

1. Introducción

El microprocesador Z80, introducido en la década de 1970, es una unidad central de procesamiento de 8 bits ampliamente utilizada en sistemas embebidos y computadoras de propósito general. Para que un microprocesador funcione, requiere estar interconectado a través de buses de direcciones, datos y control con memorias y dispositivos de entrada y salida (E/S).

El objetivo de este proyecto es estructurar un mapa de memoria específico que ubique el código a partir de la dirección 0000h y los datos en los bloques superiores, decodificando de forma correcta las señales de control para la activación de periféricos a partir de la dirección 40h. Finalmente, se demuestra la operatividad del hardware propuesto mediante un programa capaz de procesar y validar caracteres ASCII introducidos por el usuario.

2. Funcionamiento del sistema

2.1. Microprocesador Z80

El Z80 es un procesador con un bus de datos bidireccional de 8 bits y un bus de direcciones de 16 bits, lo que le permite direccionar directamente hasta 64 Kbytes de memoria (0000h a FFFFh) y 256 puertos de E/S. Sus señales de control principales incluyen /MREQ (petición de memoria), /IORQ (petición de E/S), /RD (lectura) y /WR (escritura), las cuales deben combinarse mediante lógica externa para gestionar la lectura y escritura en los dispositivos conectados.

2.2. Memorias

- 28C256 (EEPROM): Memoria no volátil con capacidad de 32 Kbytes. Al requerir solo 16 Kbytes para el código, el pin de dirección superior (A14) se conecta a tierra para aprovechar únicamente la mitad inferior de su capacidad.
- 6264 (SRAM): Memoria volátil estática de 8 Kbytes. Se utilizan dos circuitos integrados para sumar los 16 Kbytes de almacenamiento de datos requeridos por las especificaciones del diseño.

2.3. Interfaz para periféricos

Se utiliza el circuito 8255 PPI (Programmable Peripheral Interface). Este chip proporciona 24 pines de E/S programables organizados en tres puertos de 8 bits (Puerto A, Puerto B y Puerto C). Como el sistema requiere 6 puertos en total, se integran dos chips 8255 decodificados en el mapa de E/S del procesador, permitiendo una comunicación paralela eficiente con dispositivos externos como teclados o pantallas.

3. Diseño del sistema

3.1. Mapa de memoria y E/S

- Memoria de Código (EEPROM 28C256): Inicia en 0000h y termina en 3FFFh (16 Kbytes).
- Memoria de Datos 1 (SRAM 6264): Inicia en B800h y termina en D7FFh (8 Kbytes).
- Memoria de Datos 2 (SRAM 6264): Inicia en D800h y termina en F7FFh (8 Kbytes), logrando que la memoria culmine adyacente a la dirección F800h solicitada.
- Puertos (8255 1 y 2): Mapeados utilizando la señal /IORQ a partir de la dirección 40h.

3.2. Diagrama electrónico

El esquemático fue diseñado en la plataforma KiCad. El bus de datos (D0-D7) y direcciones (A0-A15) se conectan en paralelo desde el microprocesador a todos los periféricos. La selección de los chips (señal /CS) se gestiona mediante compuertas lógicas y decodificadores (como el 74LS138) que evalúan los bits superiores del bus de direcciones. Las señales /OE (habilitación de salida) y /WE (habilitación de escritura) de las memorias se obtienen combinando lógicamente las salidas /MREQ, /RD y /WR del procesador Z80.

3.3. Estructura de programación

El software principal es un intérprete BASIC ensamblado sobre el procesador Z80 a partir de un código nativo. Una vez ejecutado, el usuario introduce un programa de alto nivel en BASIC que imprime cadenas de texto en pantalla, captura datos en la variable N\$, aplica un bucle FOR para analizar el código ASCII de cada carácter con la función MID\$() y detecta si es una letra (ASCII 65-90 y 97-122) o un espacio, emitiendo un mensaje de error mediante una condición IF ante cualquier anomalía.

4. Análisis de resultados

La simulación del sistema permitió comprobar el funcionamiento conjunto del microprocesador Z80, las memorias y las interfaces de entrada y salida. La ejecución del programa permite presentar solicitudes de información al usuario y recibir una cadena de caracteres para su procesamiento.

Durante la ejecución, el programa analiza cada carácter introducido mediante su código ASCII. Los caracteres correspondientes a letras mayúsculas (65-90) y minúsculas (97-122) son considerados válidos. Los espacios, identificados mediante el código 20h, se excluyen del conteo de letras, permitiendo procesar nombres y apellidos separados por espacios.

Cuando la cadena contiene únicamente caracteres permitidos, el algoritmo continúa con el procesamiento y obtiene el número de caracteres alfabéticos introducidos. Esto permite comprobar la comunicación entre el programa y el usuario, así como la correcta ejecución de las instrucciones de control del intérprete BASIC.

En caso de introducir números o símbolos que no pertenecen a los rangos ASCII establecidos, la condición de validación detecta la anomalía y el programa genera el mensaje ERROR: Caracter invalido tecleado. De esta manera, el sistema evita continuar con una cadena que no cumple las restricciones definidas.

Los resultados obtenidos permiten verificar que la organización del mapa de memoria, la conexión de los dispositivos y la lógica de control son compatibles con el funcionamiento requerido por el sistema. Asimismo, la validación de caracteres demuestra la interacción entre el software y los recursos de hardware disponibles.

5. Conclusión

El diseño e implementación del sistema Z80 demostró la importancia de una correcta estructuración en el mapa de memoria para evitar colisiones en los buses de direcciones. Se comprobó que el acoplamiento del procesador con memorias y periféricos programables requiere un trabajo riguroso en la lógica de control. Adicionalmente, el desarrollo de rutinas y algoritmos de validación

(ensamblador y alto nivel) resalta la capacidad de este hardware para interactuar de forma condicionada con usuarios, logrando cumplir de manera integral las especificaciones impuestas de almacenamiento, conectividad paralela y procesamiento de datos.

Referencias

- [1] Zilog, "Z80 Microprocessor Family User Manual," Zilog Inc., UM008011-0816, 2016.
- [2] Atmel, "AT28C256 256K (32K x 8) Paged Parallel EEPROM Datasheet," Atmel Corporation, Rev. 0006M, 2012.
- [3] Renesas, "HM6264A 8192-word x 8-bit High Speed CMOS Static RAM," Renesas Electronics, Rev. 2.0, 1999.
- [4] Intel, "8255A Programmable Peripheral Interface," Intel Corporation, 1990.